

OWUS

 [maoyab/OWUS](#)

 english

 gemini-2.5-flash

 7bb8fe8

 Dec 9, 2025

Chapters

Overview of OWUS

Chapter 1: Ecohydrological Data Preparation

Chapter 2: Stochastic Soil Water Balance Model (SSWM)

Chapter 3: Water Use Strategy Framework

Chapter 4: Model Parameter Inference (MCMC)

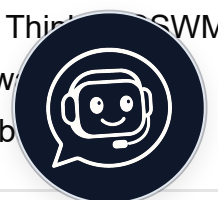
Chapter 5: Scientific Visualization

Show Raw Markdown

Chapter 2: Stochastic Soil Water Balance Model (SSWM)

Welcome back! In [Chapter 1: Ecohydrological Data Preparation](#), we learned how to gather and organize all the vital ecohydrological ingredients – like rainfall, temperature, and soil type – for our study sites. We made sure everything was clean, consistent, and ready for action.

Now that our ingredients are prepped, it's time to put them to use! This chapter introduces the "chef" of our OWUS project: the **Stochastic Soil Water Balance Model (SSWM)**. Think of SSWM as the main engine that takes all that prepared data and figures out exactly how water moves through the soil and plants. It's like a detailed calculator for a plant's entire water budget.



What Problem Does SSWM Solve? The Plant's Water Budget!

Imagine a plant living in a specific spot. Every day, it receives water from rain, it drinks water from the soil through its roots, and some water evaporates directly from the soil surface or from its leaves. This constant give-and-take of water determines how much water is available in the soil for the plant to thrive.

The central problem: We want to understand and predict:

1. **How much water is typically in the soil?** Is it usually wet, or often dry?
2. **How much water do plants use?** Do they use a lot or a little?
3. **How does this vary over time and under different conditions?**

The **SSWM** is designed to answer these questions. It's not just a simple calculation; it's a smart model that considers the *chances* of different events (like rainfall) to give us a complete picture of the soil water situation.

A concrete example: Let's say you're studying the 'AT-Neu' grassland site from Chapter 1. You want to know, on average, how much soil moisture is typically available there, and how much water the grass uses during its growing season. The SSWM will use the site's unique rainfall patterns, soil properties, and plant characteristics to simulate this "water budget" over the long term.

Key Concepts: Understanding Water's Journey

The SSWM tracks water in and out of the soil. Let's break down the main components:

1. Soil Water: The Plant's Pantry

The soil acts like a water storage tank. Plants can only access water within certain limits:

- **Saturated (Full Tank):** When the soil has as much water as it can possibly hold. Any extra water will drain away or run off.
- **Field Capacity (Comfortable):** The amount of water held in the soil after excess water has drained away. This is generally a comfortable level for plants.
- **Wilting Point (Emptying):** The point where plants can no longer extract water from the soil, even if some water is still technically present. Plants will start to wilt.
- **Permanent Wilting Point (Completely Empty):** Below this, plants can't recover.

The SSWM calculates the *probability* of the soil being at various levels between these points.

Instead of a single value, it gives you a "distribution" – showing how often the soil is likely to be wet, medium, or dry over a long period.

2. Water Inflow: Rainfall

The primary way water enters the soil is through rainfall. The "Stochastic" part of SSWM comes into play here: rainfall is a random process. We don't know *exactly* when and how much it will rain, but we know the *probability* of rainfall events based on historical data. The model uses this probability to simulate realistic water inputs.

3. Water Outflow: Evapotranspiration (ET)

Water leaves the soil-plant system mainly through:

- **Transpiration (T)**: This is water "drunk" by the plant roots and released as vapor from its leaves. It's essential for plant growth and cooling.
- **Evaporation (E)**: This is water evaporating directly from the soil surface, especially after rain and before the canopy closes.

The SSWM also accounts for **drainage** (water moving deep into the soil below the root zone) and **runoff** (water flowing over the surface).

How to Use SSWM: Calculating the Water Budget

To use the SSWM, you need to provide it with the site-specific parameters that we prepared in [Chapter 1: Ecohydrological Data Preparation](#). These parameters describe the soil, the plant, and the climate for your study site.

The SSWM in `OWUS` is implemented as a Python class called `SM_C_H`. You'll initialize this class with your site's parameters, and then it will calculate the long-term water balance characteristics.

Here's how you would typically use it:

```
1 import pandas as pd
2 import pickle # Used to load saved parameters
3 from sswm import SM_C_H # This is our SSWM model!
4
5 # 1. Load the prepared parameters for 'AT-Neu' from Chapter 1
6 site_id = 'AT-Neu'
7 params_filepath = f'DATA/WUS/WUS_input_data/{site_id}_parameters.pickle'
8
9 with open(params_filepath, 'rb') as f:
10     site_params = pickle.load(f)
```

```

11
12 # 2. Convert parameters to a list (as expected by the model)
13 # The order of parameters is crucial, as defined in sswm.py
14 # This list is simplified for demonstration; the actual list is longer.
15 model_input_params = [
16     site_params['Td'],          # Duration of a time step (e.g., 1 day)
17     site_params['rf_alpha'],    # Rainfall frequency parameter
18     site_params['rf_lambda'],   # Rainfall intensity parameter
19     site_params['ETo'],         # Potential evapotranspiration
20     site_params['LAI'],         # Leaf Area Index
21     site_params['RAI'],         # Root Area Index
22     site_params['hc'],          # Canopy height
23     site_params['Zm'],          # Rooting depth
24     site_params['Ps0'],         # Saturated soil matric potential
25     site_params['b'],           # Brooks-Corey exponent (soil texture)
26     site_params['Ks'],          # Saturated hydraulic conductivity
27     site_params['n'],           # Soil porosity
28     site_params['s_h'],         # Permanent wilting point
29     site_params['s_fc'],        # Field capacity
30     site_params['k_xl_max'],    # Maximum xylem conductivity
31     site_params['Px50'],        # Xylem vulnerability parameter
32     site_params['Pg50'],        # Stomatal sensitivity parameter
33     site_params['T_GS'],        # Total growing season duration
34 ]
35
36 # 3. Initialize the SSWM model with these parameters
37 print(f"Initializing SSWM for site: {site_id}")
38 sswm_model = SM_C_H(model_input_params)
39 print(f"SSWM initialized successfully!")
40
41 # 4. Access key outputs (after initialization, results are pre-calculated)
42 # These represent long-term averages or properties
43 print(f"Mean Transpiration (mm/day): {sswm_model.get_mean_t(sswm_model.p0):.2f}")
44 print(f"Mean Evapotranspiration (mm/day): {sswm_model.get_mean_et(sswm_model.p0):.2f}")
45 print(f"Mean water stress (0=no stress, 1=full stress): {sswm_model.get_mean_stress(sswm_model.p0):.2f}")
46
47 # The model also calculates epsilon, an efficiency metric for water use
48 if not np.isnan(sswm_model.epsilon):
49     print(f"Water Use Efficiency (epsilon): {sswm_model.epsilon:.2f}")
50 else:
51     print("Water Use Efficiency (epsilon) could not be calculated (model constrained)")
52

```

What happens here?

- We load the `site_params` dictionary created in Chapter 1.
- We then carefully extract the specific values needed by the `SM_C_H` model and put them into a list. The order is important!
- When `SM_C_H(model_input_params)` is called, the model *immediately* calculates the steady-state (long-term average) conditions for soil moisture, plant water use, and water stress based on these inputs.
- We then print out some of the key results, like the average transpiration and evapotranspiration rates, and a measure of how stressed the plants typically are.

Under the Hood: How SSWM Works

The `SM_C_H` class in `sswm.py` is the heart of this chapter. It's a sophisticated calculator that, once given the site parameters, runs a series of complex equations to figure out the long-term water balance.

Here's a simplified flow of what happens when you initialize `SM_C_H`:

```
sequenceDiagram
    participant User
    participant SSWM_Class as SSWM Model (SM_C_H)
    participant SiteParams as Prepared Parameters
    participant Calculator as Internal Calculator

    User->>SSWM_Class: Initialize(site_params_list)
    SSWM_Class->>SiteParams: Load individual parameters (e.g., soil_b, plant
    Note over SSWM_Class: Store all site-specific details internally.
    SSWM_Class->>Calculator: Calculate baseline plant hydraulic properties (l
    SSWM_Class->>Calculator: Calculate "well-watered" transpiration (T_ww)
    Note over SSWM_Class: This is the max water use when soil is full.
    SSWM_Class->>Calculator: Determine critical soil moisture points (s_wilt
    Note over SSWM_Class: Wilting point, stress threshold, field capacity.
    SSWM_Class->>Calculator: Calculate water loss functions (Evaporation, Tr
    Note over SSWM_Class: How much water leaves at different soil moisture l
    SSWM_Class->>Calculator: Compute soil moisture probability distribution
    Note over SSWM_Class: The "stochastic" part: how likely is each soil moi
    SSWM_Class->>Calculator: Calculate mean Transpiration (get_mean_t), ET (
    SSWM_Class-->>User: Initialization Complete (results ready for access)
```

Diving Deeper into the Code (`sswm.py`)

Let's look at some key parts of the `SM_C_H` class (simplified for clarity).

1. **Initialization (`__init__`):** This is where all the magic starts. It takes your `params` list and uses it to set up all the necessary variables and run the core calculations.

```
1 # From sswm.py (simplified)
2 class SM_C_H(object):
3     def __init__(self, params, nbins=100, q=1, f_wilt=0.05, f_star=0.95, constrain=1):
4         # Store general model settings
5         self.nbins = nbins # Number of soil moisture bins for probability distribution
6
7         # Unpack the site-specific parameters (like the ingredients from Chapter 2)
8         [self.T_GS, self.rf_alpha, self.rf_lambda, self.Eo, self.Td,
9          self.LAI, self.RAI, self.hc, self.Zm,
10         self.Ps0, self.b, self.Ks, self.n, self.s_h, self.s_fc,
11         self.k_xl_max, self.Px50, self.Pg50,] = params
12
13         # --- Core calculations run automatically during initialization ---
14
15         # Calculate maximum plant and soil-root conductivities
16         self.K_p_max = self.scaled_K_p_max()
17         self.K_sr_max = self.get_K_sr_max()
18
19         # Determine key soil moisture thresholds (wilting, stress, field capacity)
20         self.psi_wilt = self.psi_s_from_beta(self.f_wilt)
21         self.s_wilt = self.pot_to_s_BC(self.psi_wilt)
22         # ... and s_star, s_fc (adjusted if needed) ...
23
24         # Define potential evaporation and well-watered transpiration
25         self.E_max = self.phiE * self.Eo / (self.n * self.Zm)
26         self.T_ww = (1 - self.phiE) * self.beta_ww * self.Eo / (self.n * self.Zm)
27
28         # Calculate the steady-state probability distribution of soil moisture (p0)
29         self.p0 = self.get_p0()
30
31         # Calculate mean annual Transpiration, ET, and Stress from p0
32         self.mean_tf = self.get_mean_t(self.p0) / self.rf
33         # ... and other mean values like mean_et, mean_stress, epsilon ...
34
```

```

35         # Handle cases where model cannot be solved (e.g., if parameters are unre
36         if (self.et_ok == 0) or (self.epsilon <=0) or (self.AA > 1) or (self.beta
37             # Set key results to NaN, indicating an invalid solution
38             self.epsilon = np.nan
39

```

This `__init__` method is very busy! It takes all the raw parameters and immediately starts processing them to figure out the plant's water relations, soil properties, and ultimately, the long-term averages of water fluxes and stress. Notice how `self.p0` (the probability distribution) is calculated, and then used to derive `self.mean_tf` (mean transpiration fraction) and other important metrics.

2. Water Loss Function (`loss`): This function determines how much water leaves the system (through evaporation, transpiration, and drainage) at any given soil moisture level (`s`). The amount of water lost changes depending on how wet the soil is and whether the plant is stressed.

```

1  # From sswm.py (simplified)
2  def loss(self, s):
3      # s is the soil moisture content (from 0 to 1)
4
5      if s > self.s_fc: # If soil is wetter than field capacity
6          # Max E + Max T + Drainage (excess water drains)
7          return self.E_max + self.T_ww + self.m * (np.exp(self.bb * (s - self.s_fc)
8      elif (s > self.s_star) and (s <= self.s_fc): # Soil is comfortable, no stress
9          # E decreases as soil dries, T is at max
10         return self.E_max * (s - self.s_h) / (self.s_fc - self.s_h) + self.T_ww
11     elif (s > self.s_wilt) and (s <= self.s_star): # Soil is getting dry, plant s
12         # Both E and T decrease as soil dries
13         return self.E_max * (s - self.s_h) / (self.s_fc - self.s_h) \
14             + self.T_ww * (s - self.s_wilt) / (self.s_star - self.s_wilt)
15     elif (s > self.s_h) and (s <= self.s_wilt): # Soil is very dry, plant is seve
16         # Only E occurs, T is zero
17         return self.E_max * (s - self.s_h) / (self.s_fc - self.s_h)
18     else: # Soil is at or below wilting point
19         return 0 # No water loss
20

```

This `loss` function is central because it mathematically describes how water leaves the system under different soil moisture conditions. This dynamic is key to figuring out the overall water balance.

3. Soil Moisture Probability (`get_p0`): This function calculates `p0`, which is the long-term, steady-state probability distribution of soil moisture. Essentially, it tells you, "Over a long period, what percentage of the time is the soil likely to be at a certain wetness level?"

```
1 # From sswm.py (simplified)
2 def get_p0(self):
3     # Create a range of possible soil moisture values (bins)
4     s_list = np.linspace(0., 1., (self.nbins + 1))
5
6     # Calculate the probability for each soil moisture bin
7     # The __p0 method (not shown here) contains complex equations
8     # that link rainfall, water loss, and soil properties to probability.
9     p0 = np.array([self.__p0(s) for s in s_list])
10
11     # Normalize the probabilities so they sum to 1
12     c = np.sum(p0)
13     return p0 / c
14
```

The `p0` array is a fundamental output of the stochastic model. It doesn't give you a single soil moisture value, but rather a *range* of possible values and their likelihoods. This probabilistic approach is what makes the SSWM so powerful for understanding long-term ecohydrological patterns.

4. Mean Water Fluxes (`get_mean_t`, `get_mean_et`): Once `p0` is known, we can calculate the long-term *average* transpiration, evapotranspiration, and water stress by summing up the water loss at each soil moisture level, weighted by its probability.

```
1 # From sswm.py (simplified)
2 def get_mean_t(self, p0):
3     # Tww is the maximum transpiration for the site (calculated earlier)
4     Tww = self.T_ww * self.n * self.Zm # Convert to actual units
5
6     def __T_losses(s):
7         # This nested function describes how transpiration changes with soil mois
8         if (s > self.s_star): # No stress, max T
```

```

9         return Tww
10        elif (s > self.s_wilt) and (s <= self.s_star): # Stress, T decreases
11            return Tww * (s - self.s_wilt) / (self.s_star - self.s_wilt)
12        else: # Severe stress, no T
13            return 0
14
15        # Create a list of soil moisture values
16        s_list = np.linspace(0., 1., len(p0))
17
18        # Calculate transpiration for each soil moisture level
19        T_at_s = np.array([__T_losses(s) for s in s_list])
20
21        # Multiply by the probability distribution (p0) and sum to get the mean
22        mean_T = T_at_s * p0
23        return np.sum(mean_T)
24
25    def get_mean_et(self, p0):
26        # Similar logic as get_mean_t, but includes both evaporation and transpiration
27        # It uses the full 'loss' function (or a specific ET loss function)
28        # weighted by p0 to calculate the mean evapotranspiration.
29        # ... (implementation details omitted for brevity) ...
30        return np.sum(ET_at_s * p0)
31

```

These functions show how the `p0` (the probability distribution) is used to find the *average* transpiration and evapotranspiration over the long term. This is a powerful result, as it doesn't rely on simulating every single day, but rather on the underlying statistics of the system.

Conclusion

In this chapter, we've explored the heart of the `OWUS` project: the **Stochastic Soil Water Balance Model (SSWM)**. We learned that it acts as a sophisticated calculator for a plant's water budget, taking our carefully prepared ecohydrological data and simulating how water flows through the soil and plants. We discovered that the "stochastic" aspect means it considers the probabilities of events, providing a long-term understanding of soil moisture distribution and average water use.

By understanding the SSWM, you now know how `OWUS` can predict critical ecohydrological variables like mean transpiration and soil moisture stress. But plants don't just use water; they have *strategies* for how they manage it.

Are you ready to discover how plants adapt their water use based on their environment? Then let's move on to the next chapter!

Next Chapter: Water Use Strategy Framework

Generated by [AI Codebase Knowledge Builder](#). **References:** [1], [2]

© 2025 Codebase to Tutorial. All rights reserved.

[Terms of Service](#) [Privacy Policy](#)